

COP 1047C – Introduction to Python Programming

Chapter 4 - Branching & Decision Logic

4.1 – If Statements

A **branch** executes only when a condition is True.

```
discount = 0
if user_age > 65:
    discount = 20
    user_type = "Senior"
# The following statement is outside of the if block
total_price = cost * (100-discount)/100
```

- ✓ Condition evaluates to True or False
- ✓ Indentation defines the block

If-Else Structure

Exactly ONE branch executes.

```
if age < 25:  
    insurance_price = 4800 # Executed if True  
else:  
    insurance_price = 2200 # Executed if False
```

Never both. Never neither.

4.2 – Equality Operators

Operator	Meaning
==	Equal
!=	Not equal

⚠ Use `==` not `=` inside conditions.

```
if num_years == 50:  
    print("Golden anniversary!")
```

Multi-Branch (if-elif-else)

Only ONE branch executes.

```
if num_sales == 0:
    bonus = 0
elif num_sales == 1:
    bonus = 2
else:
    bonus = 10
#
print( f"{bonus=}" ) # This statement is immediately
                    # outside of the if-elif-else
```

Evaluated top → bottom.

4.3 & 4.4 – Detecting Ranges

Implicit range detection:

```
if age < 16:  
    print("Too young")  
elif age < 25:  
    print("16-24")  
elif age < 40:  
    print("25-39")  
else:  
    print("40+")
```

Each later condition assumes previous were False.

Relational Operators

Operator	Meaning
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal

⚠ `=<` and `<>` are invalid.

4.5 – Logical Operators

Operator	Meaning
and	Both True
or	At least one True
not	Reverse Boolean

Example:

```
if 18 <= age <= 65:
```

Equivalent to:

```
if age >= 18 and age <= 65:
```

Booleans and logical operators

A Boolean refers to a value that is either True or False. Note that True and False are keywords in Python and must be capitalized. A programmer can assign a Boolean value by specifying True or False, or by evaluating an expression that yields a Boolean.

Figure 4.5.1: Creating a Boolean.

```
my_bool = True    # Assigns my_bool with the boolean value True  
is_small = 4 < 3  # Assigns is_small with the result of the expression 4 < 3 (False)
```

Keywords and, or, and not (lowercase) are used to represent the AND, OR, and NOT logical operators. Logical operators are commonly used in expressions of if-else statements.

Table 4.5.2: Logical operators.

Logical operator	Description
a and b	Boolean AND: True when both operands are True.
a or b	Boolean OR: True when at least one operand is True.
not a	Boolean NOT (opposite): True when the single operand is False (and False when operand is True).

Ranges with Gaps

```
if age <= 12:  
    price = 11  
elif age >= 65:  
    price = 12  
else:  
    price = 14
```

Gap: 13–64

4.6 – Explicit Ranges with OR

```
if (office_num >= 100 and office_num <= 150)  
    or (office_num >= 200 and office_num <= 250):  
    print("Valid office")
```

4.7 – Multiple DISTINCT if Statements

These are independent:

```
if apples < 20:  
    boxes = 3  
  
if apples < 10:  
    boxes -= 1
```

⚠ Different from if-elif!

Multiple blocks may execute.

Nested If Statements

```
sales_type = 2
sales_bonus = 5

if sales_type == 2:
    if sales_bonus < 5:
        sales_bonus = 10
    else:
        sales_bonus += 2
#
# What is the value of sales_bonus?
```

Used when decisions depend on prior decisions.

4.8 – Comparing Data Types

- ✓ Integers compare arithmetically
- ✓ Strings compare lexicographically
- ⚠ Floats should NOT use == reliably

```
0.1 + 0.2 == 0.3    # Often False
```

Use tolerance comparisons instead.

Common Errors

- ✗ Using = instead of ==
- ✗ Invalid operators like => or <>
- ✗ Comparing incompatible types

```
1 < "abc" # TypeError
```

4.9 – Membership Operators

in / not in

```
numbers = [5, 6, 7]

if 6 in numbers:
    print("Found")
```

Works with:

- Lists
- Strings
- Tuples
- Dictionaries (checks keys!)

Dictionary Membership

```
my_dict = {"A": 1, "B": 2}

"B" in my_dict    # True
2 in my_dict      # False (checks keys only)
```

Identity Operators

is / is not

```
x = 1000
y = 1000

x is y      # Usually False
x == y      # True
```

- ✓ `==` compares values
- ✓ `is` compares memory identity

Avoid `is` unless checking object identity.

4.10 – Order of Evaluation

Operator precedence:

1. Parentheses
2. Arithmetic
3. Relational
4. not
5. and
6. or

Example:

```
x == 5 or y == 10 and z != 10
```

Evaluates as:

```
x == 5 or (y == 10 and z != 10)
```

Common Parentheses Error

```
not x == 3
```

Actually evaluates as:

```
not (x == 3)
```

NOT:

```
(not x) == 3
```

Use parentheses to clarify.

4.11 – Code Blocks & Indentation

Python uses indentation to define blocks.

```
if year < 1970:  
    antique = True
```

- ✓ Standard: 4 spaces
- ✗ Never mix tabs and spaces

Indentation errors = runtime failure.

Multi-Line Structures

Allowed for:

- Long strings
- Lists
- Dictionaries
- Function calls

```
my_list = [  
    1, 2, 3,  
    4, 5, 6  
]
```

Not new blocks — just formatting.

4.12 – Conditional Expressions (Ternary)

General form:

```
expr_true if condition else expr_false
```

Example:

```
y = 5 * x if x == 2 else 9 * x
```

Condition evaluated first.

Equivalent to the following:

```
if x == 2:  
    y = 5 * x  
else:  
    y = 9 * x
```

When to Use Conditional Expressions

Good:

```
status = "negative" if value < 0 else "non-negative"
```

Avoid for complex logic — readability matters.

Chapter 4 Master Takeaways

- ✓ Conditions evaluate to True/False
- ✓ Indentation defines structure
- ✓ Only ONE branch executes in if-elif
- ✓ Multiple if statements may all execute
- ✓ Use logical operators carefully
- ✓ Understand precedence rules
- ✓ Prefer clarity over cleverness

End of Chapter 4

Questions?
